

```

In [1]: # =====
# Notebook setup
# =====

%load_ext autoreload
%autoreload 2

# Control figure size
interactive_figures = False
if interactive_figures:
    # Normal behavior
    %matplotlib widget
    figsize=(9, 3)
else:
    # PDF export behavior
    figsize=(14, 4)

from util import util
import os
import numpy as np
from tensorflow.keras import callbacks
import warnings
warnings.simplefilter("ignore")

# Load data
data_folder = os.path.join '..', 'data')
data = util.load_data(data_folder)

# Identify input columns
dt_in = list(data.columns[3:-1]) # Exclude metadata

# Focus on a subset of the data
data_by_src = util.split_by_field(data, field='src')
dt = data_by_src['train_FD004']

# Split training and test machines
tr_ratio = 0.75
np.random.seed(42)
machines = dt.machine.unique()
np.random.shuffle(machines)

sep = int(tr_ratio * len(machines))
tr_mcn = machines[:sep]
ts_mcn = machines[sep:]

tr, ts = util.partition_by_machine(dt, tr_mcn)

# Standardization and normalization
trmean = tr[dt_in].mean()
trstd = tr[dt_in].std().replace(to_replace=0, value=1) # handle static fields
ts_s = ts.copy()
ts_s[dt_in] = (ts_s[dt_in] - trmean) / trstd
tr_s = tr.copy()
tr_s[dt_in] = (tr_s[dt_in] - trmean) / trstd

```

```

trmaxrul = tr['rul'].max()
ts_s['rul'] = ts['rul'] / trmaxrul
tr_s['rul'] = tr['rul'] / trmaxrul

# Define a cost model
failtimes = dt.groupby('machine')['cycle'].max()
safe_interval = failtimes.min()
maintenance_cost = failtimes.max()
cmodel = util.RULCostModel(maintenance_cost=maintenance_cost, safe_interval=

```

RUL Prediction as Classification

RUL Prediction as Classification

RUL-based maintenance can also be tackled *using a classifier*

- We build a classifier to determine whether a failure will occur in ε steps
- We stop as soon as the classifier outputs (say) a 0, i.e.

$$f_{\varepsilon}(x, \theta) = 0$$

- f is the classifier, with parameter vector θ
- ε is the horizon for detecting a failure

In a sense, we are trying to learn *directly* a maintenance policy

- The policy is the form "stop ε units before a failure"
- The classifier tries to learn it

Classifier Architecture

We can therefore immediately define our classifier architecture:

```

In [2]: nn1 = util.build_nn_model(input_shape=(len(dt_in), ), output_shape=1, hidden
util.plot_nn_model(nn1)

```

Out [2]:



- Like in the regression case, we use a Multilayer Perceptron
- The only difference is the use of a sigmoid activation in the output layer
- For `hidden = []` we get Logistic Regression
- ...Which of course if going to be our first model

Training

Before training, we need to define the classes

In turn, this requires to define the detection horizon ε :

```
In [8]: class_thr = 20
tr_lbl = (tr['rul'] >= class_thr)
ts_lbl = (ts['rul'] >= class_thr)
```

- The class is "1" if a failure is more than ε steps away
- The class is "0" otherwise

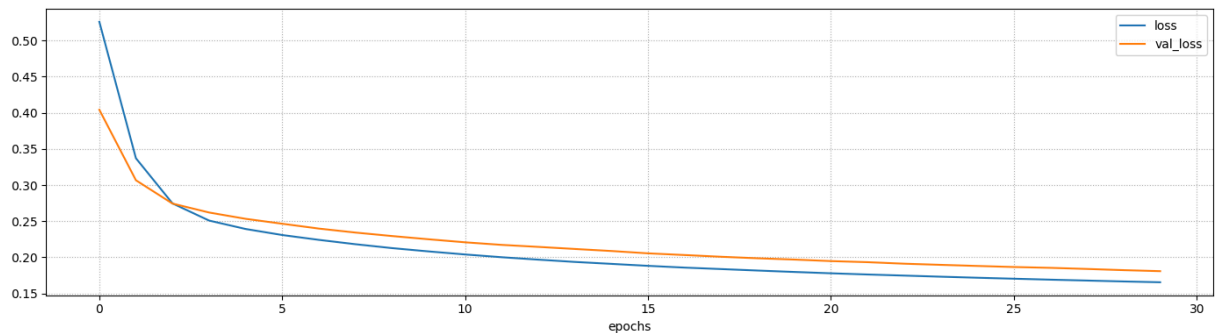
Classification problems *tend to be easier* than regression problems

- On the other hand, learning the whole policy
- ...May be trickier than just estimating the RUL

Training

Let's start by training the simplest possible model

```
In [9]: nn1 = util.build_nn_model(input_shape=(len(dt_in), ), output_shape=1, hidden
history = util.train_nn_model(nn1, tr_s[dt_in], tr_lbl, loss='binary_crossentropy',
                             verbose=0, patience=10, batch_size=32, validation_split=0.2)
util.plot_training_history(history, figsize=figsize)
```



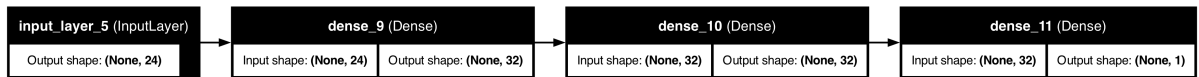
Final loss: 0.1656 (training), 0.1809 (validation)

Training

Then let's try with a deeper model

```
In [10]: nn2 = util.build_nn_model(input_shape=(len(dt_in), ), output_shape=1, hidden
util.plot_nn_model(nn2)
```

Out [10]:

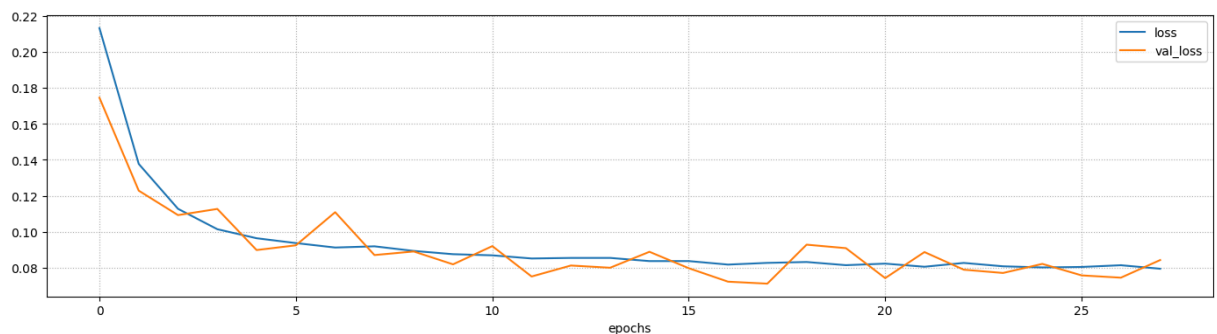


- Now we have two hidden layers
- ...Each with 32 neurons

Training

Let's train it and check the results

```
In [11]: nn2 = util.build_nn_model(input_shape=(len(dt_in), ), output_shape=1, hidden
history = util.train_nn_model(nn2, tr_s[dt_in], tr_lbl, loss='binary_crossentropy',
verbose=0, patience=10, batch_size=32, validation_split=0.2)
util.plot_training_history(history, figsize=figsize)
```



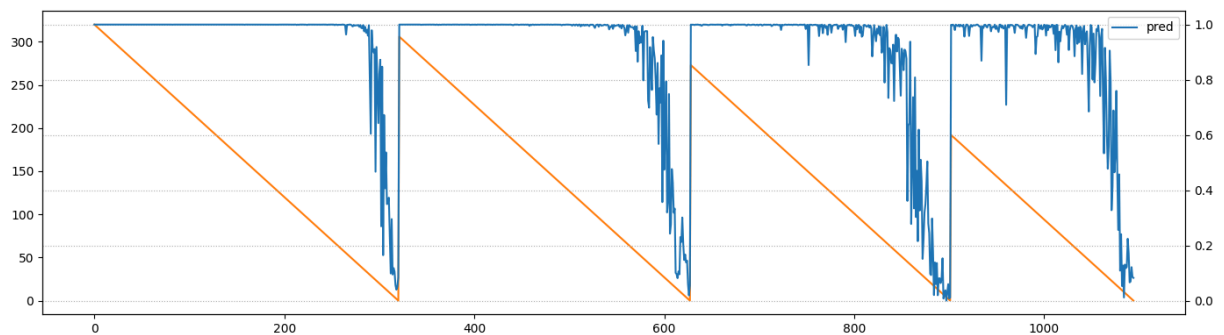
Final loss: 0.0795 (training), 0.0843 (validation)

There is a significant improvement over Logistic Regression

Predictions

The model prediction can be interpreted as a probabilities of *not* stopping

```
In [12]: tr_pred2_prob = nn2.predict(tr_s[dt_in], verbose=0).ravel()
stop = 1095
util.plot_rul(tr_pred2_prob[:stop], tr['rul'][:stop], same_scale=False, figsize=figsize)
```



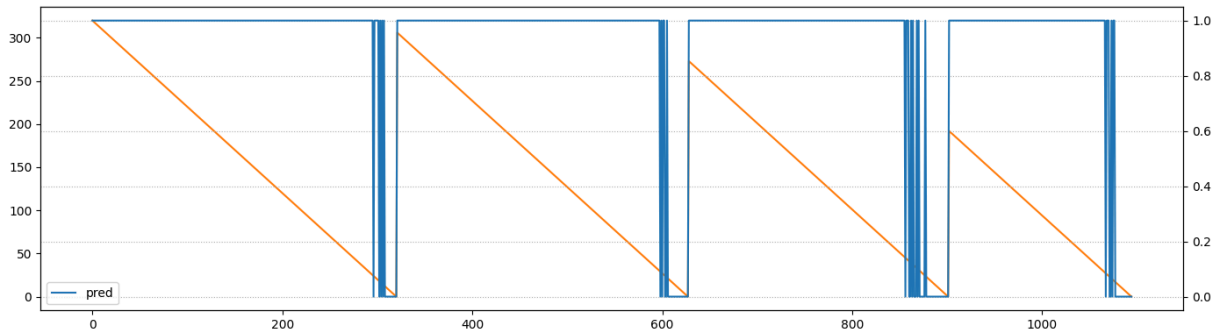
- The probability falls when closer to failures

Predictions

In practice, we'll need to convert the predictions into integers via rounding

...Unless we want to deal with one more threshold (in addition to θ)

```
In [13]: tr_pred2 = np.round(nn2.predict(tr_s[dt_in], verbose=0).ravel())  
util.plot_rul(tr_pred2[:stop], tr['rul'][:stop], same_scale=False, figsize=f
```

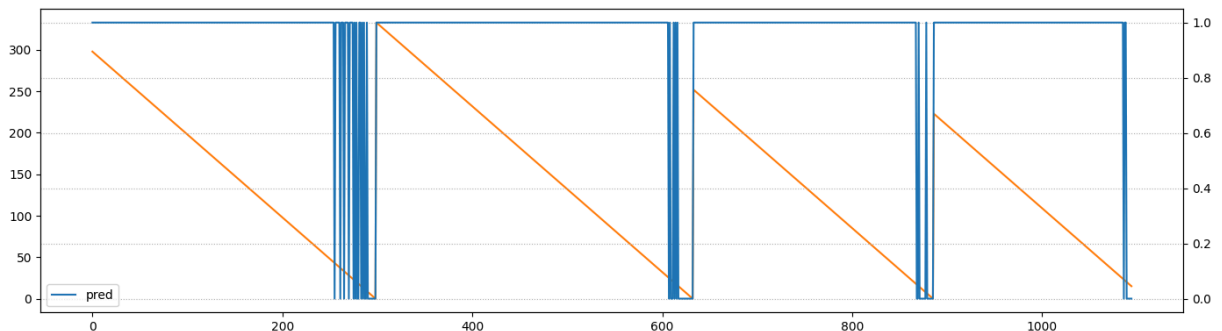


- Still, the behavior seems to be reasonable

Predictions

Let's see the behavior on the test set

```
In [14]: ts_pred2 = np.round(nn2.predict(ts_s[dt_in], verbose=0).ravel())  
util.plot_rul(ts_pred2[:stop], ts['rul'][:stop], same_scale=False, figsize=f
```



- Apparently a decent degree of generalization

Evaluation

We can evaluate the classifier directly

...Because *it defines the whole policy*, with no need for additional calibration!

- On one hand this makes this stage of the process *simpler*
- ...On the other, this is (apparently) a missed opportunity

```
In [15]: tr_c2, tr_f2, tr_s2 = cmodel.cost(tr['machine'].values, tr_pred2, 0.5, retur
ts_c2, ts_f2, ts_s2 = cmodel.cost(ts['machine'].values, ts_pred2, 0.5, retur
print(f'Cost: {tr_c2/len(tr_mcn):.2f} (training), {ts_c2/len(ts_mcn):.2f} (t
print(f'Avg. fails: {tr_f2/len(tr_mcn):.2f} (training), {ts_f2/len(ts_mcn):.
print(f'Avg. slack: {tr_s2/len(tr_mcn):.2f} (training), {ts_s2/len(ts_mcn):.
```

```
Cost: -84.14 (training), -94.02 (test)
Avg. fails: 0.00 (training), 0.00 (test)
Avg. slack: 32.43 (training), 30.40 (test)
```

- Still pretty good results, but worse than the best regression approach

Why do you think this is the case?

There are a few reasons, we will explore one

Uncalibrated Threshold

In the example from this notebook, we are defining the classes using:

```
class_thr = 20
tr_lbl = (tr['rul'] >= class_thr)
ts_lbl = (ts['rul'] >= class_thr)
```

- Like in the regression case, we are using a threshold θ
- ...But here θ is employed for defining the classes

This approach has both PROs and CONS

- PRO: we can (ideally) choose how close the failure we should stop
- CON: early signs of failure might not be evident in the chosen interval
- CON: we *did not calibrate* θ

The last point should be elaborated a bit more

Taking a Step Back

In the *regression* case, we are formally solving:

$$\operatorname{argmin}_{\varepsilon} \sum_{k \in K} \operatorname{cost}(f(x_k, \theta^*), \varepsilon) \quad (1)$$

$$\text{s.t.: } \theta^* = \operatorname{argmin}_{\theta} L(f(x_k, \theta), y_k) \quad (2)$$

- Where θ^* is the optimal parameter vector (i.e. the network weights)

- L is the loss function (i.e. the MSE), and $cost$ is our cost model
- The threshold ε is chosen so as to minimize the cost

This is a *bilevel optimization* problem

- However, since ε appears neither in L nor in f
- ...It can be *decomposed into two sequential subproblems*

Taking a Step Back

In the *classification* case, we are formally solving:

$$\operatorname{argmin}_{\varepsilon} \sum_{k \in K} cost(f(x_k, \theta^*), 1/2) \quad (3)$$

$$\text{s.t.: } \theta^* = \operatorname{argmin}_{\theta} L(f(x_k, \theta), 1_{y_k \geq \varepsilon}) \quad (4)$$

- We use a canonical threshold in the cost model (i.e. 0.5)
- L is again the loss function (binary cross entropy)
- $1_{y_k \geq \varepsilon}$ is the indicator function of $y_k \geq \varepsilon$ (i.e. our class labels)

Unlike the previous one, this problem *cannot be decomposed*

...Because ε appears in the loss function!

- This means we need to *optimize ε and θ at the same time*

Black Box Optimization

Let's sketch a possible optimization approach

1. We search over the possible values of ε
2. For the given ε value, we compute $1_{y_k \geq \varepsilon}$ (i.e. the class labels)
3. We train the model to compute θ^*
4. Then we compute the cost
5. ...And finally we repeat, for the next value of ε

At the end of the process, we choose the configuration with the best cost

In principle we could use grid search again, but...

- Evaluating the cost is *slow*, since it requires retraining
- The search space is *grows exponentially* with the number of parameters

We need a better optimization method!